05506026 Digital Image Processing (Java)

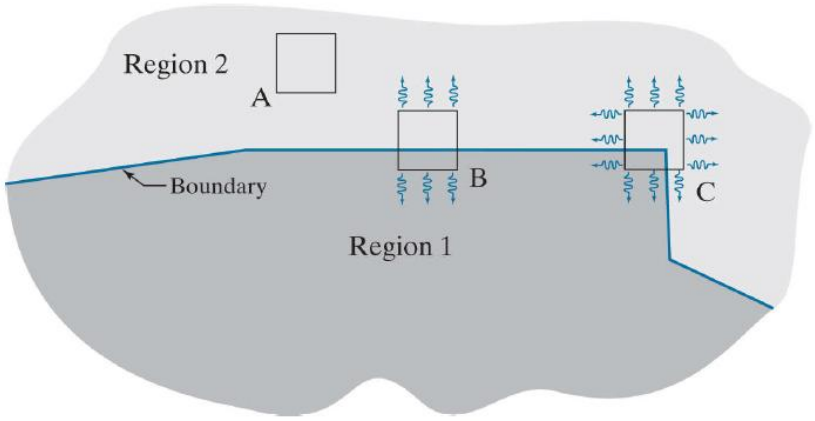
Lab 12

Objectives

1. Understand how to perform Harris-Stephens corner detection algorithm.

Harris-Stephens corner detection algorithm

Harris-Stephens corner detection algorithm, or simply called Harris corner detector, is a popular method introduced in 1988 used in computer vision to identify points in an image where the intensity changes significantly in multiple directions. These points, known as **corners** or **interest points** or **feature points**, are useful for various applications like object recognition, image matching, and motion tracking. The main idea of this algorithm is to find pixels where their intensity changes significantly in multiple directions.



The first step of this algorithm is to compute the image gradients in the x and y directions, denoted as I_x and I_y. These gradients represent the rate of change of pixel intensity in the horizontal and vertical directions. We will use central difference approximation to determine the first derivative here. The products of the gradients which are I_x^2, I_y^2, and $I_x \cdot I_y$ will also be computed. These products will be used in the matrix in further steps.

ImageManager.java

```
class ImageManager
{
    ...
    public ArrayList<Point> detectHarrisFeatures (int strongest)
    {
        // convert to gray scale first
        convertToGrayscale();

        double[][] Ix = new double[height][width];
        double[][] Iy = new double[height][width];

        // Initialize matrices to store products of gradients
        double[][] Ix2 = new double[height][width];
        double[][] Iy2 = new double[height][width];
        double[][] Ixy = new double[height][width];

        // Compute gradients Ix and Iy, drop the border
        for (int y = 1; y < height - 1; y++)
        {
            for (int x = 1; x < width - 1; x++)
            {
                int color = img.getRGB(x, y);
                int gray = color & 0xff;
                Ix[y][x] = ((img.getRGB(x + 1, y) & 0xff) - (img.getRGB(x - 1, y) & 0xff)) /
2.0;
                Iy[y][x] = ((img.getRGB(x, y + 1) & 0xff) - (img.getRGB(x, y - 1) & 0xff)) /
2.0;
                Ix2[y][x] = Ix[y][x] * Ix[y][x];
                Iy2[y][x] = Iy[y][x] * Iy[y][x];
                Ixy[y][x] = Ix[y][x] * Iy[y][x];
            }
```

The second step is to smooth all products using Gaussian smoothing. This step helps to reduce noise and ensures that the corner response is more stable. This is because the Harris algorithm is interested only in the rate of intensity changing which can include noises. This is one of Harris algorithm's limitations.

ImageManager.java

```
class ImageManager
{
    ...
    public ArrayList<Point> detectHarrisFeatures (int strongest)
    {
        ...
        // apply 3 x 3 gaussian smoothing for each matrices
        double[][] Sx2 = new double[height][width];
        double[][] Sy2 = new double[height][width];
        double[][] Sxy = new double[height][width];

        double[] gaussian =
        {
            1.0 / 16.0, 2.0 / 16.0, 1.0 / 16.0,
            2.0 / 16.0, 4.0 / 16.0, 1.0 / 16.0,
            1.0 / 16.0, 2.0 / 16.0, 1.0 / 16.0
        };

        for (int y = 1; y < height - 1; y++)
        {
            for (int x = 1; x < width - 1; x++)
            {

                Sx2[y][x] = 0;
                Sy2[y][x] = 0;
                Sxy[y][x] = 0;

                for (int i = y - 1; i <= y + 1; i++)
                {
                    for (int j = x - 1; j <= x + 1; j++)
                    {
                        Sx2[y][x] += Ix2[i][j] * gaussian[(i - (y - 1)) * 3 + (j - (x - 1))];
                        Sy2[y][x] += Iy2[i][j] * gaussian[(i - (y - 1)) * 3 + (j - (x - 1))];
                        Sxy[y][x] += Ixy[i][j] * gaussian[(i - (y - 1)) * 3 + (j - (x - 1))];
                    }
                }
            }
        }
```

The third step is to compute the second-moment matrix. This matrix captures the distribution of gradients in a local neighbourhood which can analyse the intensity variations of different directions around a pixel.

$$M = \begin{bmatrix} S_{xx} & S_{xy} \\ S_{xy} & S_{yy} \end{bmatrix}$$

This brings two values to the account: -

- Determinant of the matrix which tells how much the picture changes in **all** directions combined. If the determinant is large, it means there's a lot of change happening in both directions (a corner).

$$\det(M) = S_{xx} \cdot S_{yy} - S_{xy}^2$$

- Trace of the matrix represents the sum of the eigenvalues of the matrix, which gives an overall measure of the intensity change in the neighbourhood. A large trace suggests that there is a strong gradient, but it doesn't distinguish between corners and edges.

$$\text{trace}(M) = S_{xx} + S_{yy}$$

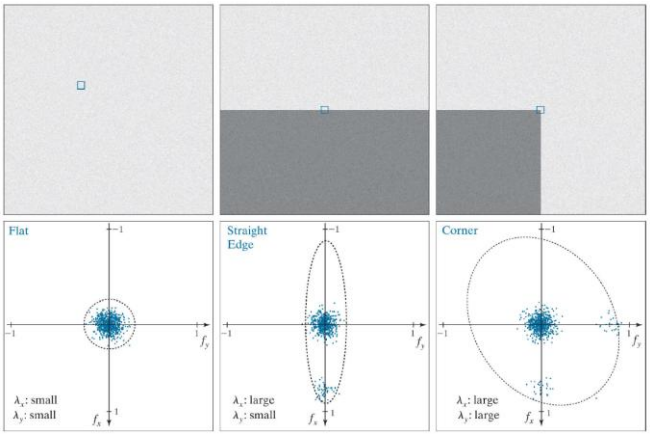
By using these two values together, we can compute the corner response in function R.

$$R = \det(M) - k \cdot \big(\text{trace}(M)\big)^2$$

k is a constant of a small value which usually between 0.04 and 0.06.

The value from R tells us what the pixel is: -

- If the value is high positive, the pixel is a corner.
- If the value is low positive, the pixel is a flat area.
- If the value is negative, the pixel is an edge.



ImageManager.java

```
class ImageManager
{
    ...
    public ArrayList<Point> detectHarrisFeatures (int strongest)
    {
        ...
        double[][] corners = new double[height][width];

        // Compute the corner response function R
        // High R = Corner, Low R = Flat, Negative R = Edge
        for (int y = 0; y < height; y++)
        {
            for (int x = 0; x < width; x++)
            {
                double det = Sx2[y][x] * Sy2[y][x] - Sxy[y][x] * Sxy[y][x];
                double trace = Sx2[y][x] + Sy2[y][x];
                corners[y][x] = det - 0.04 * trace * trace;
            }
        }
```

The last step is to check if each value is a local maximum (the largest value comparing to its neighbours). If it is, then we will keep the value in a list. We will also keep tracking the number of strongest values in the list according to the received parameters. Finally, we will mark those strongest corners with red X marks and return those corners for further usages.

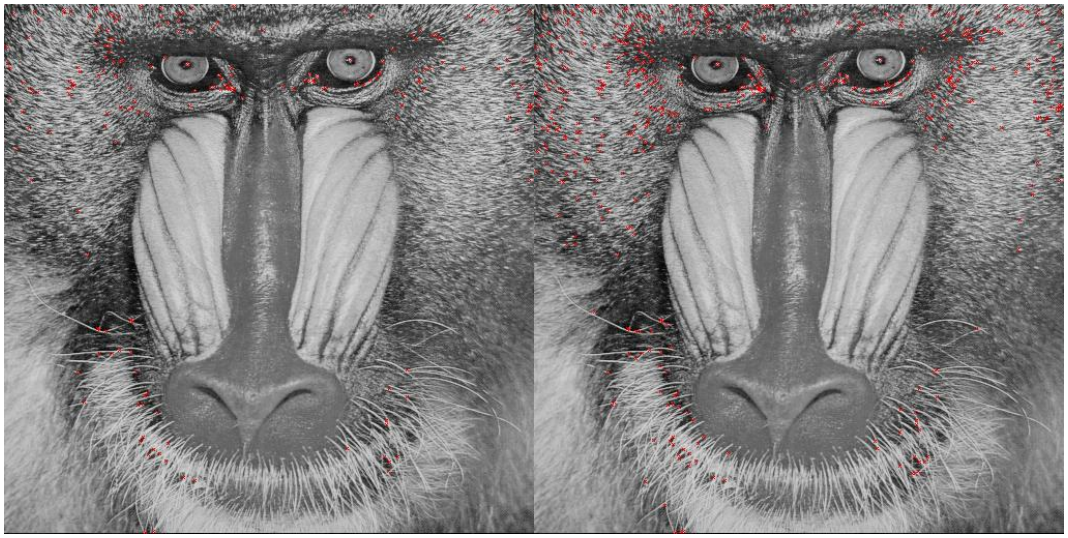
ImageManager.java

```
class ImageManager
{
    ...
    public ArrayList<Point> detectHarrisFeatures (int strongest)
    {
        ...
        ArrayList<Point> cornerPoints = new ArrayList<>();
        ArrayList<Double> cornerValues = new ArrayList<>();

        // Maxima Suspression (see if it is the maximum value to the neighbours)
        for (int y = 1; y < height - 1; y++)
        {
            for (int x = 1; x < width - 1; x++)
            {
```

```
                // if negative, not a corner
                if (corners[y][x] < 0) continue;
                // see if local maxima
                double peak = corners[y][x];
                boolean isMaxima = true;

                // Check 3x3 neighborhood
                for (int k = -1; k <= 1 && isMaxima; k++)
                {
                    for (int l = -1; l <= 1; l++)
                    {
                        if (k == 0 && l == 0) continue; // Skip the center pixel

                        int testX = x + k;
                        int testY = y + l;

                        if (corners[testY][testX] > peak)
                        {
                            isMaxima = false;
                            break; // Early exit if a larger neighbor is found
                        }
                    }
                }
                if (isMaxima) {
                    // Point is a local maxima, find the correct position to insert
                    int insertPos = 0;
                    while (insertPos < cornerValues.size() && cornerValues.get(insertPos) >
peak)
                    {
                        insertPos++;
                    }

                    // Insert corner in the correct position
                    cornerPoints.add(insertPos, new Point(x, y));
                    cornerValues.add(insertPos, peak);

                    // If we have more points than needed, remove the weakest ones
                    if (cornerPoints.size() > strongest)
                    {
                        cornerPoints.remove(strongest);
                        cornerValues.remove(strongest);
                    }
                }
            }
        }
```

```
        restoreToOriginal();
        convertToGrayscale();

        // Draw red X
        for (Point p : cornerPoints)
        {
            int redColor = (255 << 16) | (0 << 8) | 0;
            img.setRGB(p.x, p.y, redColor);
            img.setRGB(p.x + 1, p.y + 1, redColor);
            img.setRGB(p.x + 1, p.y - 1, redColor);
            img.setRGB(p.x - 1, p.y + 1, redColor);
            img.setRGB(p.x - 1, p.y - 1, redColor);
        }

        return cornerPoints;
    }
```

The result of 200 and 500 strongest corner values using Harris corner detector of mandril.bmp is as follow.



Quest 016

Implement the method of Harris-Stephens corner detection algorithm on mandril.bmp and select 1,000 and 2,000 strongest corners.

The player who finishes this task will be awarded **20 EXP** and **2 Moves.**

Deadline: 7 September 2025